

AI-Powered Book Publishing System

Retrieval-Augmented Generation (RAG) Framework

Internal Technical Proposal

Objective

The objective of this project is to design and implement a lightweight, AI-driven internal system for a small team (4–5 users) to automate book content generation, editing, and management using Retrieval-Augmented Generation (RAG).

The focus is on efficiency, cost optimization, and ease of use while maintaining high-quality output.

Project Overview

The system will enable users to:

- Aggregate and process multiple data sources (books, research papers, articles)
- Generate structured, human-like book content
- Interact with content via conversational interface
- Edit and regenerate sections dynamically
- Manage documents and workflows via a unified dashboard

Core Features

Content Ingestion

- Multi-format document upload (PDF, DOCX, text)
- Automated parsing and intelligent chunking
- Metadata extraction and tagging

AI Content Generation

- Multi-source context fusion
- Structured book creation (chapters, sections)
- Prompt-based customization (tone, style, length)

Interactive Editing

- Chat-based querying of documents
- Section-level regeneration
- Context-aware rewriting

Plagiarism Reduction

- Embedding-based similarity detection
- Semantic rewriting pipeline

Dashboard

- Document lifecycle management
- Version tracking
- Project-level organization

System Architecture

- **Backend Layer:** Python-based API using FastAPI
- **AI Pipeline:** RAG architecture (retrieval + generation)
- **Embedding Layer:** Transformer-based vector encoding
- **Vector Storage:** ChromaDB (lightweight, local-first)
- **Storage:** Local filesystem / AWS S3
- **Frontend (Optional):** Minimal UI for interaction

Technical Stack

Component	Technology
Core Language	Python
AI Frameworks	LangChain, LlamaIndex
LLMs	OpenAI GPT (primary), Ollama (optional local models)
Embedding Models	OpenAI Embeddings, Sentence Transformers
Vector Database	ChromaDB
Document Processing	PyMuPDF, pdfplumber, Unstructured.io, spaCy
Backend	FastAPI, Uvicorn

Database	SQLite / PostgreSQL
Storage	Local / AWS S3
Deployment	Docker, Local / Cloud (AWS EC2)

AI Workflow

1. Document ingestion and preprocessing
2. Text chunking and embedding generation
3. Storage in vector database
4. Context retrieval using similarity search
5. Content generation via LLM
6. Interactive editing and refinement

Cost Estimation

Monthly Operational Cost

Component	Cost ()
LLM API Usage	3,000 – 8,000
Embedding API	1,000 – 2,000
Hosting	2,000 – 4,000
Storage	300 – 800
Database	500
Total	9,000 – 12,000 / month

Development Investment

This project will be developed by a dedicated team of 4 developers over a 3-month timeline, covering end-to-end implementation including AI pipelines, backend systems, and dashboard development.

The exact commercial terms will be finalized through discussion based on the project scope, requirements, and priorities.

Proposed Payment Structure:

- Initial milestone: Project kickoff and requirement finalization
- Mid milestone: Core RAG system completion
- Final milestone: Full delivery and deployment

Note: The final payment amount will be mutually agreed upon based on detailed discussions and finalized requirements.

Pricing References

- OpenAI API Pricing (GPT models, embeddings):
<https://openai.com/pricing>
- AWS Cloud Pricing (compute, storage):
<https://aws.amazon.com/pricing/>
- ChromaDB (open-source, no direct cost; hosting dependent)
- PostgreSQL / SQLite (open-source; hosting cost only)

Costing Assumptions

- Limited usage by 4–5 users
- Moderate document size and processing frequency
- API usage optimized via caching and chunking
- No heavy concurrent workloads

Notes

- Costs may vary based on actual usage patterns and scaling
- API costs (LLMs) are the primary variable component
- Further optimization can reduce cost by using local models (Ollama)

Development Roadmap (12 Weeks / 3 Months)

Phase / Week	Activities
Week 1	Requirement gathering, workflow analysis, and system design planning
Week 2	Finalization of architecture (RAG pipeline, storage, APIs) and selection of models and tools
Week 3	Development of document ingestion system (PDF, DOCX, articles upload)
Week 4	Text extraction, preprocessing, chunking strategies, and metadata tagging implementation
Week 5	Embedding generation pipeline and integration with vector database (ChromaDB)

Week 6	Implementation of semantic search and retrieval system (cosine similarity, indexing optimization)
Week 7	Core RAG pipeline development and prompt engineering for context-aware generation
Week 8	AI book generation engine (multi-source fusion, structured output generation)
Week 9	Development of chat interface for interactive querying and content editing
Week 10	Implementation of plagiarism reduction (semantic filtering, paraphrasing, deduplication)
Week 11	Dashboard development (document management, tracking, version control) and system integration
Week 12	End-to-end testing, performance optimization, cost tuning, and deployment

Assumptions & Constraints

- Limited user base (4–5 internal users)
- Moderate document volume
- Controlled API usage for cost efficiency
- No requirement for large-scale distributed systems

Performance & Optimization Strategy

- Efficient chunking to reduce token usage
- Caching frequent queries
- Hybrid model usage (API + local models)
- Lazy loading for document processing

Key Benefits

- Faster content production
- Reduced manual workload
- Consistent and structured output
- Improved collaboration efficiency

Conclusion

The proposed system delivers a practical, cost-efficient, and scalable AI-powered solution tailored for internal use. It balances advanced AI capabilities with simplicity, ensuring effective adoption and long-term usability.

Recommended for implementation.